YOLO26 Object Tracking

YOLO26 Object Tracking

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Course Content

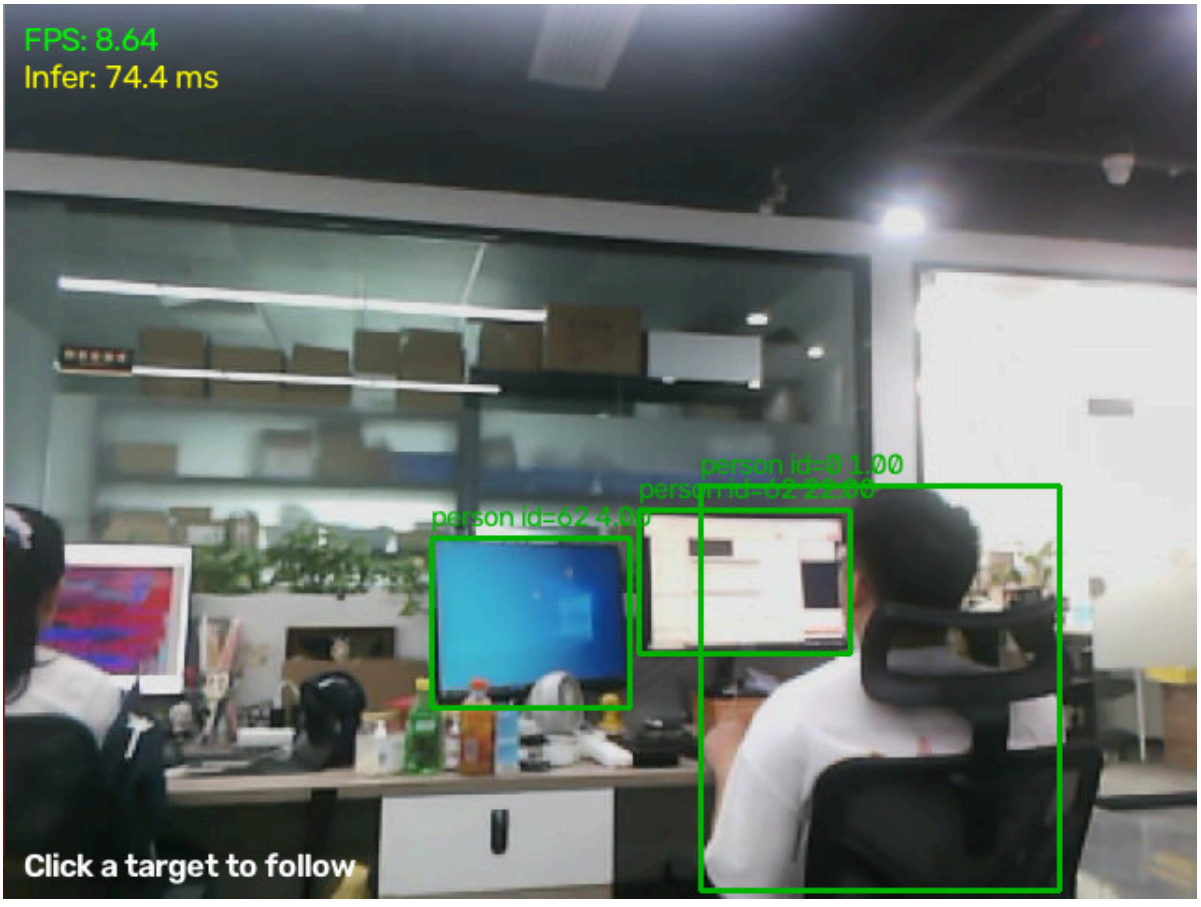
- Combining YOLO26 object detection with LiDAR forward ranging clustering, achieve stable target tracking and following for the chassis. Combine real-time LiDAR detection of the precise physical distance to the target to control forward and backward motion.

2. Preparation

```
ros2 launch yahboomcar_bringup camera_demo.launch.py demo:=yolo_track
```

3. Case Demonstration

- Visualization tracking window: every detected object in the frame is annotated with a bounding box, a class, and a unique tracking ID:



- **Left-click with the mouse** on the detection box region of the target object you want to follow. After the target is selected, the box switches to a **highlighted yellow box**, and the car immediately automatically enters the closed-loop following state!



Clear selection (re-select target)

- When you need to change the tracking target or stop the following car, while the image display window is active, press the keyboard `r` **or** `R` **key**;
- The current tracking target is cleared immediately, the car automatically publishes zero velocity to stop moving, and the status bar reverts to `Click a target to follow`; you can then click a new target to start tracking again.

4. Core Source Code Analysis

- Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_depth/yahboomcar_depth/Yolov26/yolov26_track.py
```

move_control function description:

This function receives the LiDAR range and the horizontal position of the target in the image, and computes the chassis linear velocity and angular velocity through a dual PID controller to achieve closed-loop following.

Parameter and error computation:

- `distance`: The target's physical distance (meters) measured by the LiDAR; `None` means there is currently no valid range.
- `point_x`: The horizontal pixel coordinate of the center of the target bounding box in the image.
- `image_width`: The current image width (pixels).
- `angular_error = (point_x - width * 0.5) * (640.0 / width)`: Computes the horizontal pixel error of the target deviating from the image center, normalized to a 640-width baseline; a positive value means the target is on the right, and a negative value means the target is on the left.
- `dist_error = distance - keep_dist`: Computes the difference between the current distance and the desired keep distance; a positive value means the target is too far and the car needs to move forward, and a negative value means the target is too close and the car needs to move backward.

Angular velocity control (steering):

- `angular_tolerance`: The pixel tolerance of the angular velocity control; steering is not triggered when the error is within this range.
- `angular_controller.pid_control(angular_error)`: The PID controller computes the angular velocity based on the pixel error.
- `max_angular_speed`: The upper limit of the angular velocity, preventing excessive steering speed.
- `angular_controller.reset()`: Resets the PID integral term when the error is within the tolerance, preventing accumulated deviation.

Linear velocity control (forward/backward):

- `alignment_tolerance`: The pixel tolerance of the linear velocity control; forward or backward motion is allowed only when the target is basically aligned (error less than this value).
- `start_deadband`: The start deadband; movement starts only when the distance error exceeds this value.
- `stop_deadband`: The stop deadband; movement stops when the distance error is less than this value, preventing repeated start-stop oscillation.
- `linear_motion_direction`: The motion direction state machine (0=stop, 1=forward, -1=backward), which switches states through the deadband logic to avoid frequent jitter.
- `dist_controller.pid_control(dist_error)`: The PID controller computes the linear velocity based on the distance error.
- `max_linear_speed` / `min_linear_speed`: The upper and lower limits of the linear velocity, ensuring the speed is within a reasonable range.
- `math.copysign(magnitude, direction)`: Assigns the correct direction sign to the speed magnitude output by the PID.

Side reposition logic:

- `side_reposition_threshold_px`: The side reposition pixel threshold; triggered when the target deviates from the center by more than this value and the distance is already satisfied.
- When the target is to the side but the distance is appropriate, the robot first briefly backs up (`side_reposition_reverse_speed`) to make room for turning, avoiding losing the target due to in-place rotation.

Command publishing:

- `cmd_vel_pub.publish(speed)`: Publishes the computed Twist velocity command to the chassis driver topic.

```
    def move_control(self, distance, point_x, image_width):
        '''Use laser range for linear.x and visual box position for angular.z.'''
        speed=Twist()
        width = max(1.0, float(image_width))
        angular_error = (float(point_x) - width * 0.5) * (640.0 / width)
        dist_error = (
            None
            if distance is None
            else float(distance) - float(self.keep_dist)
        )

        angular_tolerance = max(0.0, float(self.angular_pixel_tolerance))
        if abs(angular_error) > angular_tolerance:
            angular_z=self.angular_controller.pid_control(angular_error)
            max_angular = max(0.0, abs(float(self.max_angular_speed)))
            angular_z=min(angular_z,max_angular)
            angular_z=max(angular_z,-max_angular)
            speed.angular.z = angular_z
        else:
            self.angular_controller.reset()

        alignment_tolerance = max(
            angular_tolerance, float(self.linear_alignment_tolerance_px)
        )
        start_deadband = max(0.0, float(self.distance_deadband))
        stop_deadband = min(
            start_deadband,
            max(0.0, float(self.distance_stop_deadband)),
        )
        if dist_error is not None and abs(angular_error) <= alignment_tolerance:
            if self.linear_motion_direction == 0:
                if dist_error > start_deadband:
                    self.linear_motion_direction = 1
                elif dist_error < -start_deadband and self.allow_reverse:
                    self.linear_motion_direction = -1
            elif self.linear_motion_direction > 0 and dist_error <=
stop_deadband:
                self.linear_motion_direction = 0
            elif self.linear_motion_direction < 0 and (
                dist_error >= -stop_deadband or not self.allow_reverse
            ):
                self.linear_motion_direction = 0
```

```
            if self.linear_motion_direction != 0:
                linear_x = self.dist_controller.pid_control(dist_error)
                max_linear = max(0.0, abs(float(self.max_linear_speed)))
                min_linear = min(
                    max_linear,
                    max(0.0, abs(float(self.min_linear_speed))),
                )
                magnitude = min(max(abs(linear_x), min_linear), max_linear)
                speed.linear.x = math.copysign(
                    magnitude, self.linear_motion_direction
                )
            else:
                self.dist_controller.reset()
        elif dist_error is None:
            self.linear_motion_direction = 0
            self.dist_controller.reset()
        else:
            self.dist_controller.reset()

        reposition_threshold = max(
            angular_tolerance, float(self.side_reposition_threshold_px)
        )
        if (
            dist_error is not None
            and self.allow_reverse
            and self.linear_motion_direction == 0
            and abs(dist_error) <= start_deadband
            and abs(angular_error) > reposition_threshold
        ):
            reverse_speed = min(
                abs(float(self.side_reposition_reverse_speed)),
                max(0.0, abs(float(self.max_linear_speed))),
            )
            speed.linear.x = -reverse_speed
        self.cmd_vel_pub.publish(speed)
```